Breast Cancer Classification Using Classical Machine Learning Models

Basic Applied Machine Learning Project

Fatemeh Bayat

May 29, 2026

Contents

1 Project Objective

The aim of this project was to classify breast cancer samples as **malignant** or **benign** using the Breast Cancer Wisconsin Diagnostic dataset. The task is a supervised binary classification problem because each sample has numerical input features and a known diagnosis label.

The project follows a complete classical machine learning workflow: data loading and dataset inspection, exploratory data analysis (EDA), preprocessing and scale-aware modelling, stratified train-test split, interpretable non-ML baselines, model comparison using stratified cross-validation, hyperparameter tuning with GridSearchCV, final test-set evaluation, and interpretation of errors, limitations, and possible future work.

Because this is a medical classification problem, performance was not evaluated using accuracy alone. The malignant class is clinically important because a malignant sample predicted as benign represents a possible missed cancer case. Therefore, malignant-specific recall, malignant F1-score, the confusion matrix, MCC, and ROC-AUC were also emphasized.

2 Dataset Description

The project used the Breast Cancer Wisconsin Diagnostic dataset available through textttsklearn.datasets.load_breast_cancer. The dataset contains measurements computed from digitized images of fine needle aspirate (FNA) biopsies of breast masses. Each sample is represented by numerical features describing properties of cell nuclei.

2.1 Dataset Size and Variables

The dataset contains 569 samples, 30 original numerical input features, one target variable, and an additional human-readable diagnosis column added during the analysis. Therefore, after adding the target and diagnosis columns, the working dataframe contained 32 columns.

Table 1: Target label encoding used in the dataset.

Label	Diagnosis	Clinical meaning
0	Malignant	More clinically critical class
1	Benign	Non-malignant class

2.2 Feature Groups

The 30 numerical variables are based on cell nucleus properties such as radius, texture, perimeter, area, smoothness, compactness, concavity, concave points, symmetry, and fractal dimension. These properties appear as mean, standard error, and worst-value measurements.

3 Understanding the Target Labels

A key methodological point in this project was the encoding of the target variable. In this dataset, malignant is encoded as 0 and benign is encoded as 1. However, many scikit-learn metrics treat label 1 as the positive class by default. For this reason, malignant-specific metrics were computed using `pos_label=0`.

4 Exploratory Data Analysis

EDA was performed before model training to understand dataset quality, class distribution, feature ranges, and possible modelling challenges. The EDA included checking dataframe shape and data types, missing values, class balance, numerical feature ranges, selected feature distributions, and correlations between features.

4.1 Missing Values

No missing values were found, so no imputation strategy was needed and all 569 samples were kept.

4.2 Class Balance

The class distribution was moderately imbalanced, with more benign than malignant samples.

Table 2: Class distribution in the full dataset.

Class	Number of samples	Percentage
Benign	357	62.74%
Malignant	212	37.26%

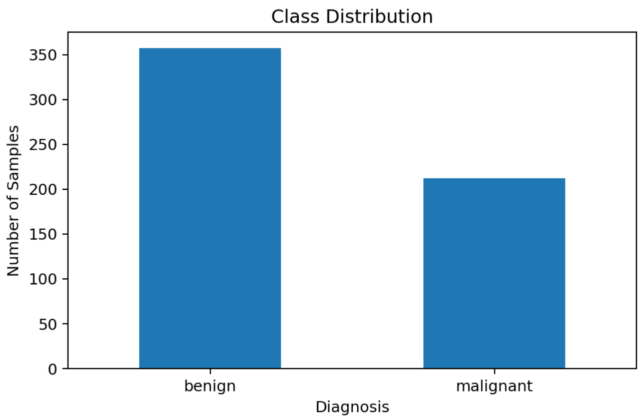

Figure 1: Class distribution of benign and malignant samples.

This imbalance shows why accuracy alone is not sufficient.

4.3 Feature Ranges and Scaling Motivation

The numerical features have very different value ranges, which matters for algorithms that depend on distances, margins, or coefficient optimization.

Table 3: Examples of feature ranges motivating scaling.

Feature	Minimum	Maximum
mean radius	6.981	28.11
mean texture	9.71	39.28
mean perimeter	43.79	188.5
mean area	143.5	2501
mean smoothness	0.05263	0.1634
mean concave points	0	0.2012
worst area	185.2	4254
worst concavity	0	1.252

The scale-sensitive models were Logistic Regression, KNN, and SVM. For these models, scaling was applied inside Pipeline objects using `StandardScaler` to reduce data leakage risk.

4.4 Feature Distribution Analysis

The project visualized selected biologically interpretable features to compare distributions between benign and malignant samples.

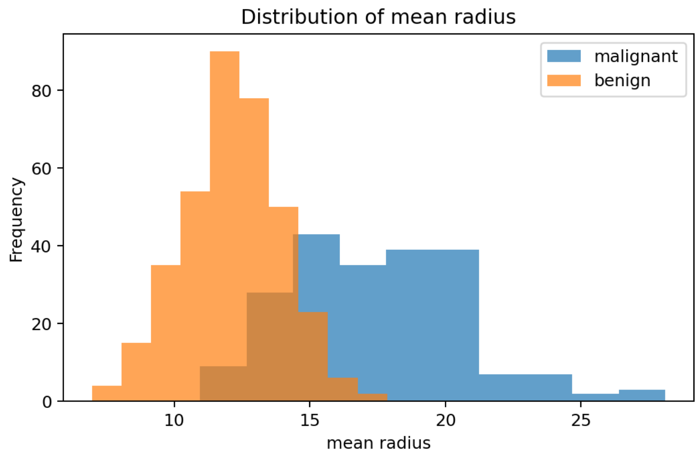

Figure 2: Distribution of mean radius by diagnosis.

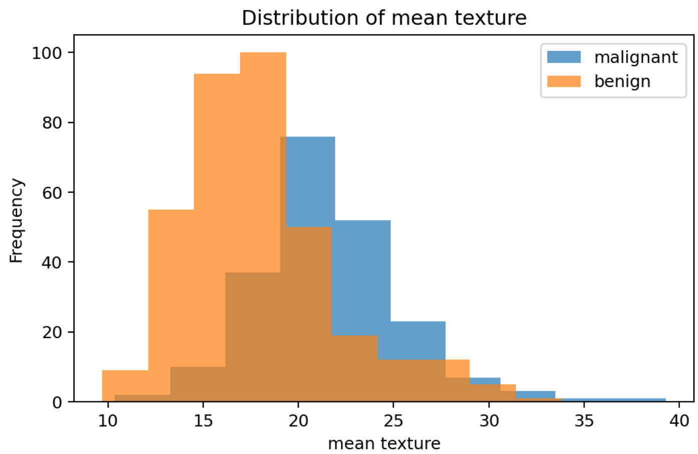

Figure 3: Distribution of mean texture by diagnosis.

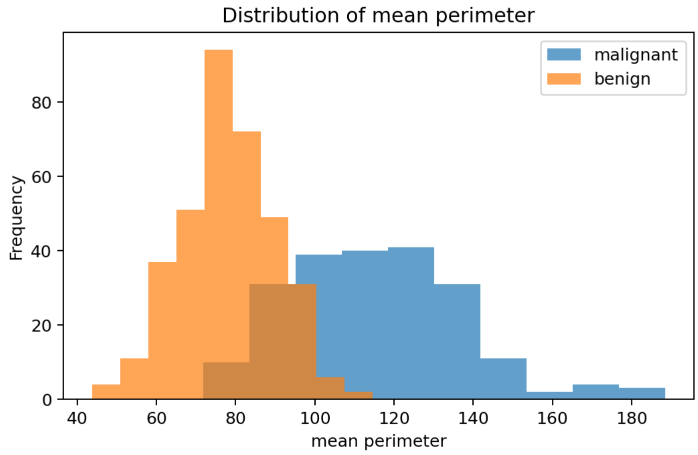

Figure 4: Distribution of mean perimeter by diagnosis.

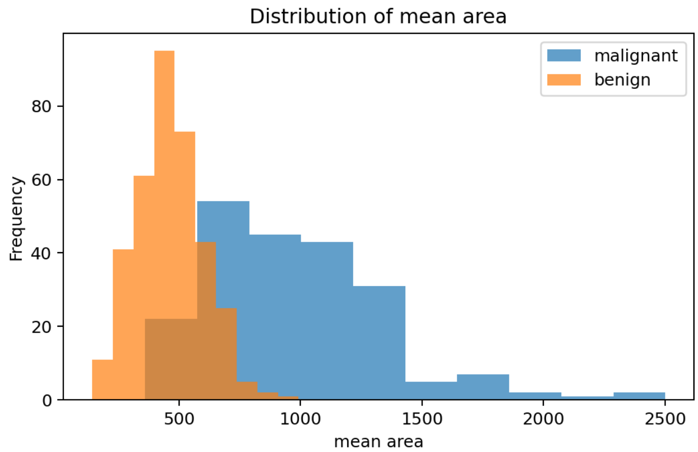

Figure 5: Distribution of mean area by diagnosis.

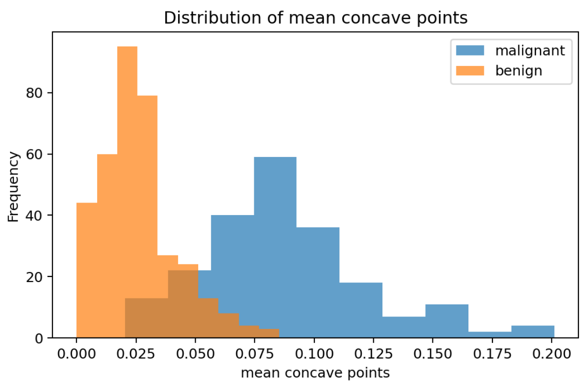

Figure 6: Distribution of mean concave points by diagnosis.

These plots show useful discriminatory information, but the overlap between classes justifies multivariate models.

4.5 Correlation Analysis

High correlation between some variables was expected because radius, perimeter, and area describe related properties.

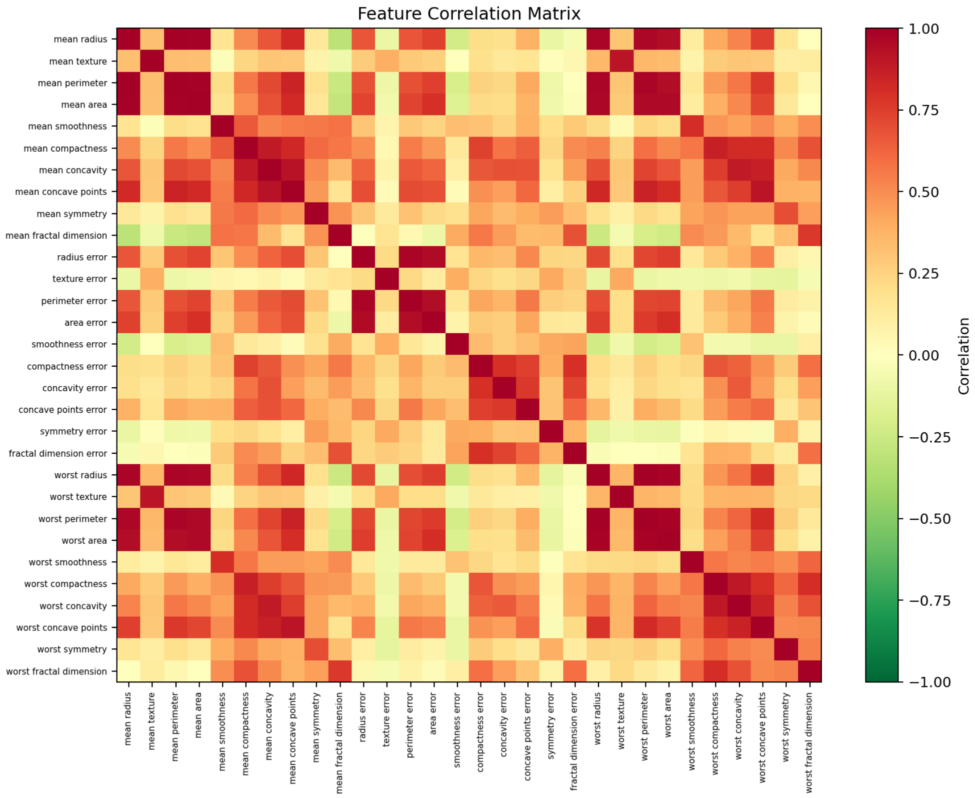

Figure 7: Correlation matrix of the 30 numerical input features.

Table 4: Examples of highly correlated feature pairs.

Feature 1	Feature 2	Correlation
Mean perimeter	Mean radius	0.9979
Worst perimeter	Worst radius	0.9937
Mean area	Mean radius	0.9874
Mean area	Mean perimeter	0.9865
Worst area	Worst radius	0.9840

Highly correlated features may provide overlapping information, but all features were initially kept because removing variables too early could remove useful predictive information.

5 Train-Test Split

Before model training, the dataset was split into training and test sets using a stratified split.

```
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.20, random_state=42, stratify=y
)
```

Table 5: Train-test split configuration.

Setting	Value
Test size	0.20
Random state	42
Stratification	Yes, by target label
Training samples	455
Test samples	114

Table 6: Class distribution after stratified splitting.

Set	Benign	Malignant
Training set	285	170
Test set	72	42

The test set was kept unseen during model comparison and hyperparameter tuning.

6 Rule-Based Non-ML Baselines

A new part of the final notebook was the addition of simple non-ML baselines. These baselines were included as interpretable reference points before training multivariate machine learning models. They were not used for final model selection; final selection was based on cross-validation, hyperparameter tuning, and final test-set evaluation.

6.1 One-Feature Rule-Based Baseline

The first rule-based baseline used only **mean concave points**. The rule predicts a sample as malignant when its mean concave points value is above a threshold. The selected threshold, chosen only from the training set, was approximately **0.0488**.

Table 7: One-feature rule-based baseline test performance.

Metric	Value
Accuracy	0.9035
Balanced accuracy	0.9137
Malignant precision	0.8163
Malignant recall	0.9524
Malignant F1-score	0.8791
MCC	0.8062

Table 8: One-feature rule-based baseline confusion matrix.

Actual / Predicted	Predicted malignant	Predicted benign
Actual malignant	40	2
Actual benign	9	63

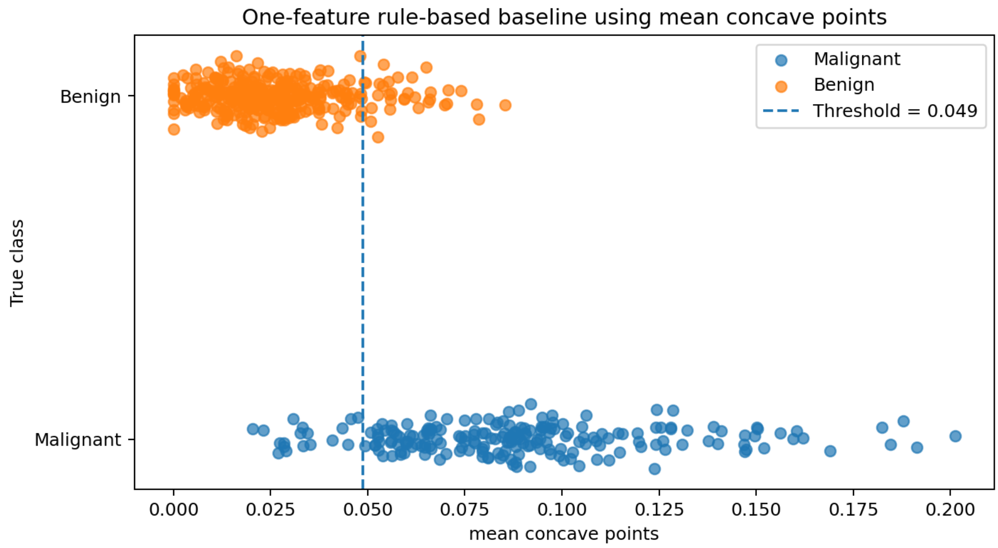

Figure 8: One-feature rule-based baseline using mean concave points.

The one-feature baseline is simple and transparent, but it uses only one feature and cannot model interactions between variables.

6.2 Optional Two-Feature Rule-Based Baseline

As an additional exploratory baseline, a simple rule using **mean concave points** and **mean area** was tested. The rule predicts malignant if either feature is above its selected threshold. The thresholds were selected using only the training set. The selected thresholds were approximately **0.0488** for mean concave points and **690.74** for mean area.

Table 9: Two-feature rule-based baseline test performance.

Metric	Value
Accuracy	0.9035
Balanced accuracy	0.9236
Malignant precision	0.7925
Malignant recall	1.0000
Malignant F1-score	0.8842
MCC	0.8194

Table 10: Two-feature rule-based baseline confusion matrix.

Actual / Predicted	Predicted malignant	Predicted benign
Actual malignant	42	0
Actual benign	11	61

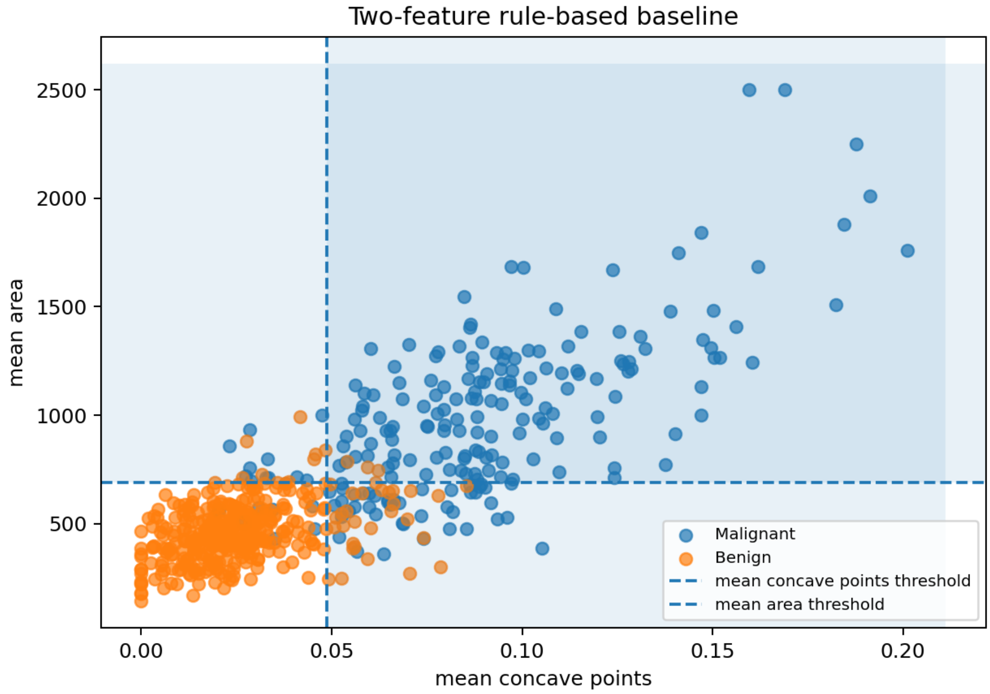

Figure 9: Optional two-feature rule-based baseline.

The two-feature rule-based baseline achieved very high malignant recall, but this came at the cost of more benign samples being incorrectly predicted as malignant. This shows the trade-off between reducing false negatives and increasing false positives.

6.3 Baseline Comparison Summary

Table 11: Comparison of rule-based baselines and the final model.

Model	Features used	Malignant recall	Main limitation
One-feature rule baseline	1	0.9524	Uses only one feature
Two-feature rule baseline	2	1.0000	More false positives
Final Logistic Regression	30	0.9524	Still has 2 false negatives

Compared with the simple baselines, the final Logistic Regression model uses all 30 numerical features and provides a better balance between malignant recall, false positives, overall accuracy, MCC, and interpretability.

7 Model Comparison with Cross-Validation

Several classical machine learning models were compared using 5-fold stratified cross-validation on the training set only.

```
cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)
malignant_recall_scorer = make_scorer(recall_score, pos_label=0)
```

Table 12: Models compared in the project.

Model	Reason for inclusion
Logistic Regression	Simple, interpretable, suitable for binary classification
KNN	Distance-based comparison model
SVM	Strong margin-based classifier, effective after scaling
Decision Tree	Interpretable non-linear model
Random Forest	Ensemble model, usually more stable than one tree

Table 13: Model comparison using 5-fold stratified cross-validation.

Model	Mean F1-macro	SD F1-macro	Mean malignant recall	SD malignant recall
Logistic Regression	0.9764	0.0106	0.9647	0.0288
Random Forest	0.9603	0.0188	0.9588	0.0300
SVM	0.9669	0.0160	0.9529	0.0399
KNN	0.9598	0.0118	0.9294	0.0235
Decision Tree	0.9119	0.0190	0.9235	0.0353

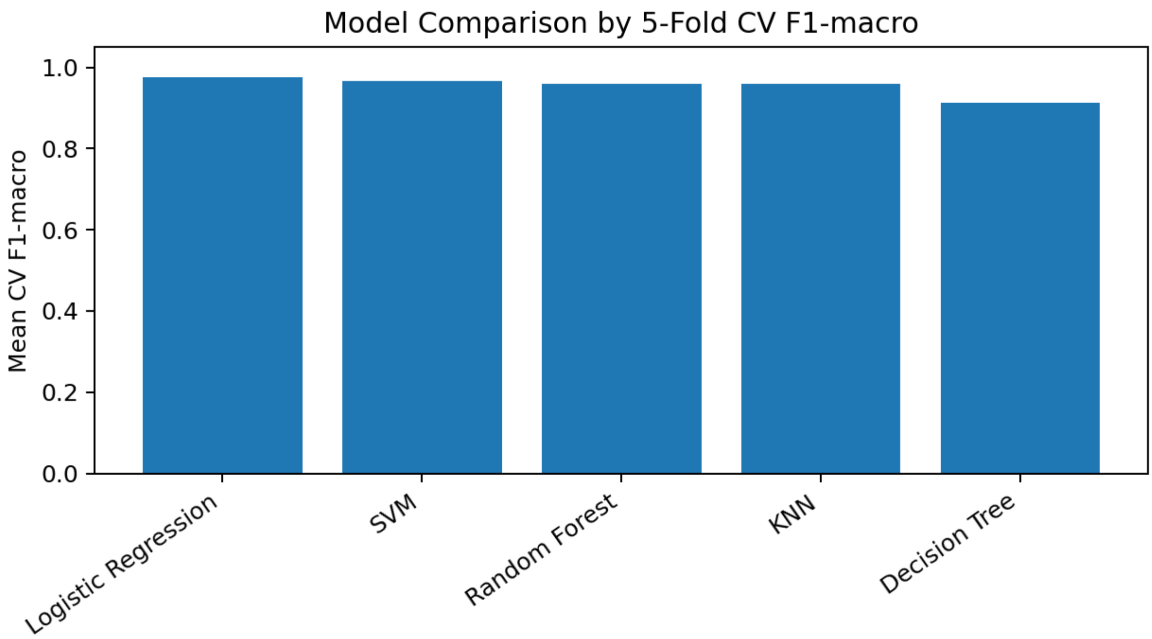

Figure 10: Model comparison by mean cross-validation F1-macro.

Logistic Regression was the strongest candidate model because it achieved the best mean F1-macro and the highest mean malignant recall in cross-validation.

8 Hyperparameter Tuning

Hyperparameter tuning was applied to Logistic Regression and SVM using `GridSearchCV` with the same stratified cross-validation object.

8.1 Logistic Regression Tuning

```
logreg_param_grid = {
    'model__C': [0.01, 0.1, 1, 10, 100],
    'model__penalty': ['l2'],
    'model__solver': ['lbfgs']
}
```

Table 14: Best Logistic Regression hyperparameters.

Parameter	Best value
C	0.1
Penalty	l2
Solver	lbfgs
Best CV F1-macro	0.9811

The value of C controls inverse regularization strength. A smaller value means stronger regularization.

8.2 SVM Tuning

```
svm_param_grid = {
    'model__C': [0.1, 1, 10, 100],
    'model__kernel': ['linear', 'rbf'],
    'model__gamma': ['scale', 'auto']
}
```

Table 15: Best SVM hyperparameters.

Parameter	Best value
C	0.1
Kernel	linear
Gamma	scale
Best CV F1-macro	0.9739

Although SVM performed well, Logistic Regression had a higher tuned cross-validation F1-macro and better interpretability.

9 Final Model Selection

The final selected model was a Logistic Regression pipeline with `StandardScaler` and `LogisticRegression(C=0`
`penalty='l2', solver='lbfgs')`. This model was selected before evaluating the test set.

10 Final Evaluation on the Test Set

The final Logistic Regression pipeline was evaluated once on the independent test set.

Table 16: Final test-set performance of Logistic Regression.

Metric	Value
Accuracy	0.9737
Balanced accuracy	0.9692
Benign precision	0.9726
Benign recall	0.9861
Benign F1-score	0.9793
Malignant precision	0.9756
Malignant recall	0.9524
Malignant F1-score	0.9639
MCC	0.9433
ROC-AUC	0.9957

The final test results were strong. However, the malignant recall of 0.9524 means that the model did not detect every malignant case.

11 Confusion Matrix and Error Analysis

Table 17: Final confusion matrix.

Actual / Predicted	Predicted malignant	Predicted benign
Actual malignant	40	2
Actual benign	1	71

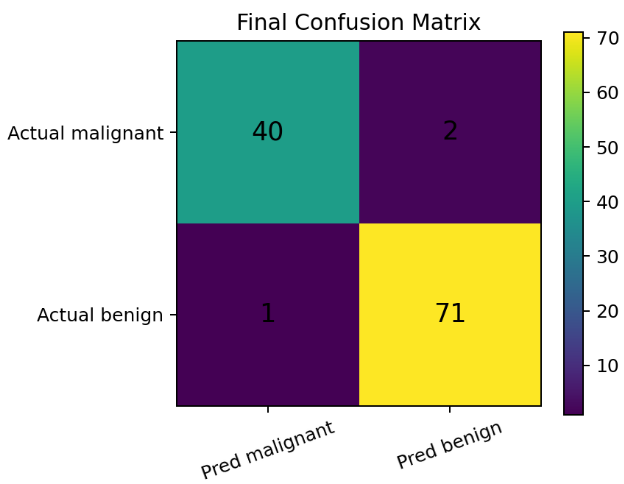

Figure 11: Final confusion matrix for the selected Logistic Regression model.

The model correctly classified 40 malignant samples and 71 benign samples. It misclassified 2 malignant samples as benign and 1 benign sample as malignant. The most clinically important error type is the malignant false negative.

12 ROC Curve

For the ROC curve, malignant was treated as the positive class.

```
y_proba_malignant = final_model.predict_proba(X_test)[:, 0]
y_test_malignant = (y_test == 0).astype(int)
final_roc_auc = roc_auc_score(y_test_malignant, y_proba_malignant)
```

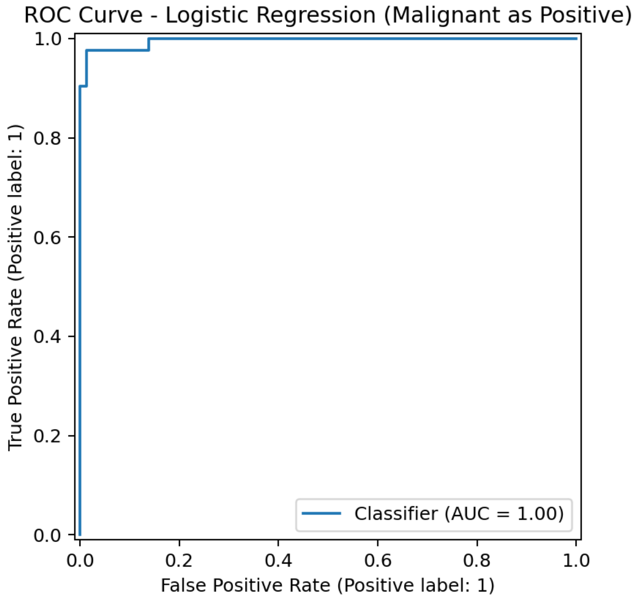

Figure 12: ROC curve for the final model, treating malignant as the positive class.

The ROC-AUC was 0.9957, indicating excellent separation between malignant and benign samples across classification thresholds. However, ROC-AUC should be interpreted together with the confusion matrix and malignant recall.

13 Conclusion

This project implemented a complete classical machine learning workflow for breast cancer classification. After EDA, preprocessing, interpretable rule-based baselines, model comparison, cross-validation, hyperparameter tuning, and independent test-set evaluation, Logistic Regression was selected as the final model. The final test results were strong: accuracy was 0.9737, balanced accuracy was 0.9692, malignant recall was 0.9524, malignant F1-score was 0.9639, MCC was 0.9433, and ROC-AUC was 0.9957.

The final model made only 3 test-set errors, but 2 of them were malignant false negatives. Therefore, the model achieved high performance but still requires careful clinical interpretation and external validation.

14 LLM Usage Declaration

LLMs were used for code debugging, language polishing, refactoring, and improving the clarity of explanations. All methodological choices, experiments, outputs, and interpretations were reviewed, checked, and validated.